

REST - Services

Table of Contents

1. Was ist HTTP?	2
1.1. Client-Server	2
1.2. URI	2
1.3. HTTP-Methoden	3
1.4. Argumentübertragung	3
1.5. HTTP-Statuscodes	4
1.6. HTTP-Request/Response	5
2. Was ist ein REST-WebService?.....	6
3. RESTEasy JAX-RS	7
3.1. Einfachen Service in Quarkus implementieren	7
3.2. Parameterübergabe	8

1. Was ist HTTP?

Hypertext Transfer Protocol (HTTP) ist ein zustandsloses Protokoll zur Übertragung von Daten auf der Anwendungsschicht über ein Netzwerk. Es stellt die Basis für das World Wide Web dar.

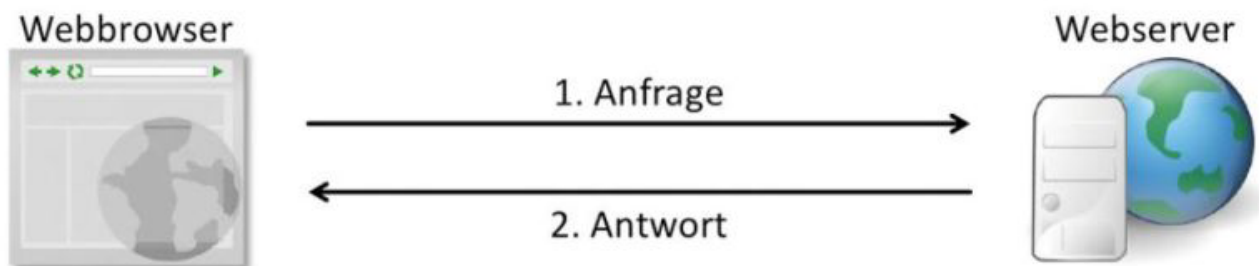
Eigenschaften:

- Zustandslos
- Verbindungslos
- Datenunabhängig (Media independent)

1.1. Client-Server

HTTP ist Client-Server basiert:

- Der HTTP-Client sendet an den Server eine Anfrage (HTTP-Request)
- Der Server schickt die entsprechende Antwort (HTTP-Response)
- Danach ist die Verbindung beendet → Keine stehende Verbindung wie bei beispielsweise Websockets!



1.2. URI

Der **Uniform Resource Identifier (URI)** ist eine Referenz auf eine Ressource im Netz, die zur Adressierung verwendet wird. Sie beschreibt das **WO**.

Aufbau eines URI:

```
URI = "protocol": "://" host [ ":" port ] [abs_path [ "?" query ]]
```

Beispiele:

```
http://localhost:8080/Service/Student?id=123
```

```
ws://172.0.0.1:8000/chat
```

1.3. HTTP-Methoden

Die HTTP-Methode gibt an, welche Aktion mit der im URI definierten Resource durchgeführt werden soll. Sie beschreibt das **WAS**.

1.3.1. GET

Wird meist zum Abfragen - also zur Übertragung von Daten zum Client - verwendet (SELECT).

1.3.2. POST

Wird meist dazu verwendet, Daten vom Client an den Server zu senden. Diese Daten werden dann oft in der DB gespeichert (INSERT)

1.3.3. PUT

Wird meist dazu verwendet, Daten am Server zu aktualisieren (UPDATE)

1.3.4. DELETE

Wird verwendet, um Daten zu löschen (DELETE)

1.3.5. Weitere Methoden

HEAD, PATCH, TRACE, OPTIONS, CONNECT

1.4. Argumentübertragung

Um Informationen an den Server zu senden stehen prinzipiell 2 Möglichkeiten zur Verfügung:

- **HTTP-GET:**

Die Daten sind Teil der URL und bleiben deshalb beim Speichern oder der Weitergabe des Links erhalten (Achtung Log-Files). Die Daten müssen URL-kodiert werden (%<Hexwert>, + für Leerzeichen)

...?Parameter1=Wert1&Parameter2=Wert2&Param3=Wert3

- **HTTP-POST:**

Übertragung der Daten im HTTP-Nachrichtenrumpf (Body), so dass diese in der URL nicht sichtbar sind.

1.5. HTTP-Statuscodes

Jede HTTP-Anfrage wird vom Server mit einem HTTP-Statuscode beantwortet. Dieser Teilt mit, ob die Anfrage erfolgreich bearbeitet wurde oder im Fehlerfall auch wo (zB Umleitungen) bzw. wie (zB mit Authentifizierung) die Information erhalten werden kann.

Übersicht über die Statuscodes:

1xx	Informationen zur Anfrage Die Bearbeitung der Anfrage dauert trotz der Rückmeldung noch an. Manchmal notwendig zur Vermeidung von Timeouts
2xx	Erfolgreiche Operation Die Anfrage wurde bearbeitet und die Antwort wird an den Anfragersteller zurückgesendet
3xx	Umleitung Zur Abfrage sind weitere Schritte des Clients notwendig, beispielsweise weil die Webseite umstrukturiert wurde und die Daten jetzt wo anders abrufbar sind.
4xx	Client-Fehler Es ist ein Fehler aufgetreten, der im Verantwortungsbereich des Clients liegt. Häufigste Vertreter sind: 404 - Dokument existiert nicht 403 - Abruf ist nicht erlaubt
5xx	Server-Fehler Es ist ein Fehler aufgetreten, der im Verantwortungsbereich des Servers liegt (Beispielsweise Exception bei der Ausführung).

1.6. HTTP-Request/Response

Beispiel GET Request:

```
GET http://localhost:37903/REST_Collection_and_DB/api/student/findAll HTTP/1.1
User-Agent: Fiddler
Host: localhost:37903
Content-Length: 0
```

Beispiel Response:

```
HTTP/1.1 200 OK
Server: Payara Server 4.1.1.161
X-Powered-By: Servlet/3.1 JSP/2.3 (Payara Server 4.1.1.161)
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: OPTIONS, GET, POST, PUT, DELETE
Access-Control-Allow-Headers: Content-Type
Content-Type: application/json
Date: Sat, 26 Nov 2016 11:17:32 GMT
Content-Length: 224

[{"matNr":"it01","name":"Anton"},
 {"matNr":"it02","name":"Berta"}
...
]
```

2. Was ist ein REST-WebService?

REST steht für **R**epresentational **S**tate **T**ransfer.

Es wird zur Kommunikation zwischen Client und Server eingesetzt. Im Gegensatz zu den komplexeren Mechanismen wie CORBA, RPC oder SOAP bietet es eine einfachere Variante, die das HTTP-Protokoll verwendet.

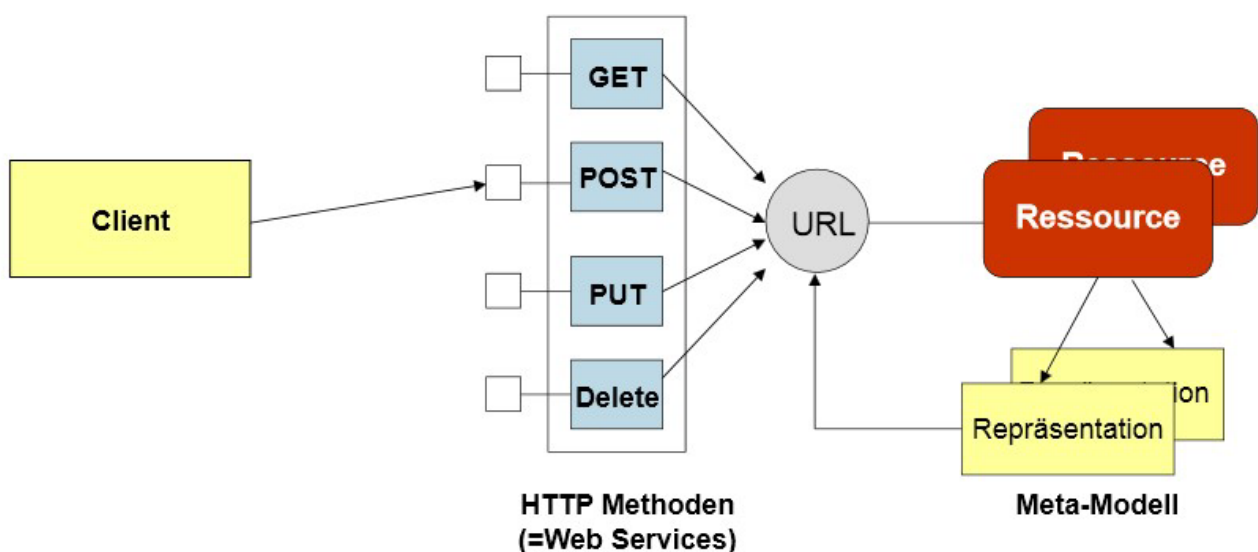
Es werden meist **nur Inhalte** übertragen, der Darstellungsteil wie im HTML entfällt üblicherweise.

Die Inhalte werden in einer strukturierten Form (meist JSON od. XML) übertragen, damit der Client od. Server diese parsen und weiterverarbeiten kann.

WebService:

- ist eine Software, die Dienste im Internet anbietet
- Unterstützt den Austausch von Informationen und Daten (B2B)
- Verwendet XML- oder JSON basierte Nachrichten
- Verwendet plattformunabhängige Protokolle
- Java: JAX-RS (REST), JAX-WS (SOAP)

Ablauf:



- Ressource = Web Seiten, Funktionen, Bilder, Dokumente etc.
- Eine Ressource kann verschiedene **Repräsentationen** haben (z.B. XML, JSON)
- Über eine **URL** eindeutig identifizierbar und lokalisierbar
- Zugriff über vier **HTTP-Methoden** (ggf. weitere Methoden) → Web Services

3. RESTEasy JAX-RS

JAX-RS steht für Java (Jakarta) API for RESTful Web Services und ist ein API, welches die Erstellung von REST APIs unterstützt.

RESTEasy ist eine Implementierung des APIs, Alternativen wären beispielsweise Jersey.

3.1. Einfachen Service in Quarkus implementieren

Hinzufügen von folgenden Extensions:

- RESTEasy JAX-RS
- RESTEasy Jackson od. RESTEasy JSON-B für Serialisierungsunterstützung

Nun kann eine Resource/Endpoint angelegt werden.

```
@Path("/demo") ①
public class DemoResource {

    @GET ②
    @Produces(MediaType.TEXT_PLAIN) ③
    @Path("hello") ④
    public String hello() {
        return "Hello RESTEasy";
    }
}
```

① Definiert den (Basis-)pfad für die Ressource

② Spezifiziert die HTTP-Methode

③ `@Produces` legt fest wie das Ergebnis geliefert werden soll (zB `MediaType.APPLICATION_JSON`). Muss mit dem Accept-Header des Requests übereinstimmen!

④ optionaler Sub-Pfad, wird an dem Basispfad von Pkt. 1 angehängt

Es ist auch möglich, komplexere Java-Klassen als Ergebnis zurückliefern zu lassen. Um dies in ein JSON-Objekt umzuwandeln ist eine entsprechende Library notwendig (Jackson, JSON-B).

Es wird dabei mittels Reflection die Struktur der Klasse ermittelt und ein entsprechendes Objekt aufgebaut.

```
@GET
@Produces(MediaType.APPLICATION_JSON) ①
@Path("person")
public Person getPerson() { ②
    return new Person(1, "Max", "Muster");
}
```

- ① Es wird angegeben, dass ein JSON-Objekt erzeugt werden soll
- ② Rückgabetyt wird auf Person gesetzt. Diese Klasse wird dann automatisch in das Json-Format gebracht

Ergebnis vom Aufruf (mittels HTTP-Testclient):

```
GET http://localhost:8080/demo/person
```

```
HTTP/1.1 200 OK
```

```
Content-Length: 50
```

```
Content-Type: application/json
```

```
{  
  "persNo": 1,  
  "firstname": "Max",  
  "lastname": "Muster"  
}
```

```
Response code: 200 (OK); Time: 1353ms; Content length: 50 bytes
```

3.2. Parameterübergabe

Parameter können über folgende Arten übergeben werden:

- PathParam
- QueryParam
- HeaderParam

Darüber hinaus können im Request-Body beispielsweise JSON-Daten übergeben werden, welche dann automatisch in Java-Objekte der jeweiligen Klasse umgewandelt werden.

Beispiel PathParam:

```
@GET  
@Produces(MediaType.TEXT_PLAIN)  
@Path("welcome/{p_firstname}/{p_lastname}") ①  
public String welcome(  
    @PathParam("p_firstname") String firstname, ②  
    @PathParam("p_lastname") String lastname) { ②  
    return String.format("Welcome %s %s!", firstname, lastname);  
}
```

- ① Im Pfad können Parameter mit geschwungenen Klammern gekennzeichnet werden
- ② Mit `@PathParam` erfolgt das Mapping des Pfad-Parameters zu dem jeweiligen Methoden-Parameter. Natürlich sind mehrere Parameter möglich

Beispiel Parameter-Objekte:

```
@POST ①
@Path("person")
@Consumes(MediaType.APPLICATION_JSON) ②
@Produces(MediaType.APPLICATION_JSON)
public Response savePerson(Person p) { ③
    System.out.println(String.format("%4d - %s %s",
        p.getPersNo(), p.getFirstname(), p.getLastname()));
    return Response.ok().entity(p).build(); ④
}
```

- ① HTTP-Methode POST
- ② Es wird ein Json-Format erwartet
- ③ Parameter ist vom Typ Person (POJO-Objekt mit Getter und Setter-Methoden)
- ④ Als Ergebnis wird ein Response-Objekt zurückgeliefert. Statuscode wird auf 200 (ok) gesetzt. Im Body wird hier zur Demonstration nochmals die Person mitgegeben (aufgrund der @Produces-Annotation wieder im Json-Format)